Integrated Smart Parking Node Report

Author: Mohin Pal | **Platform:** Wokwi Simulation | **Hardware:** ESP32 DevKit C V4

1. Executive Summary & Project Vision

This master technical document outlines an integrated system blueprint and engineering review for an **IoT-Enabled Smart Parking Node with Atmospheric Compensation**. Operating as a critical piece of modern Smart City infrastructure, the edge node uses an HC-SR04 ultrasonic core sensor to track vehicle boundaries and bay presence. To eradicate distance errors introduced by ambient air temperature and relative humidity shifts (which continuously reshape the velocity profile of sound waves in outdoor environments), an NTC thermal array dynamically adjusts calculation scales on the fly. The addition of the native ESP32 Wi-Fi layer allows telemetry streams to bridge local hardware limits, facilitating wide-area coordination over centralized cloud networks.

2. Complete Component Inventory & Blueprint Topology

The component map validated within the hardware matrix establishes clean separation between logic processing, ambient telemetry, and multi-tier audio-visual driver assistance:

Component ID	Type / Description	Smart Parking Allocation Role
`esp`	ESP32 DevKit C V4	Central Edge Microcontroller with Integrated Wi-Fi Baseband
`ultrasonic1`	HC-SR04	Ultrasonic Proximity Ranger (Gauges vehicle bumpers and layout)
`ntc1`	NTC Temperature Sensor	Tracks ambient context to compensate sonar propagation timelines
`led1`	LED (Red)	Critical Alert / Maximum Proximity Warning Indicator
`led2`	LED (Limegreen)	Bay Vacant / Approach Path Clear Indicator
`bz1`	Piezo Buzzer	Pulsed Acoustic Driver Assistant (Provides spatial backup warnings)
`logic1`	8-Ch Logic Analyzer	Records hardware TX streams for telemetry logging and validation
`r1 - r4`	Resistors (1kΩ)	Current-limiting arrays protecting visual pathways and ground nodes

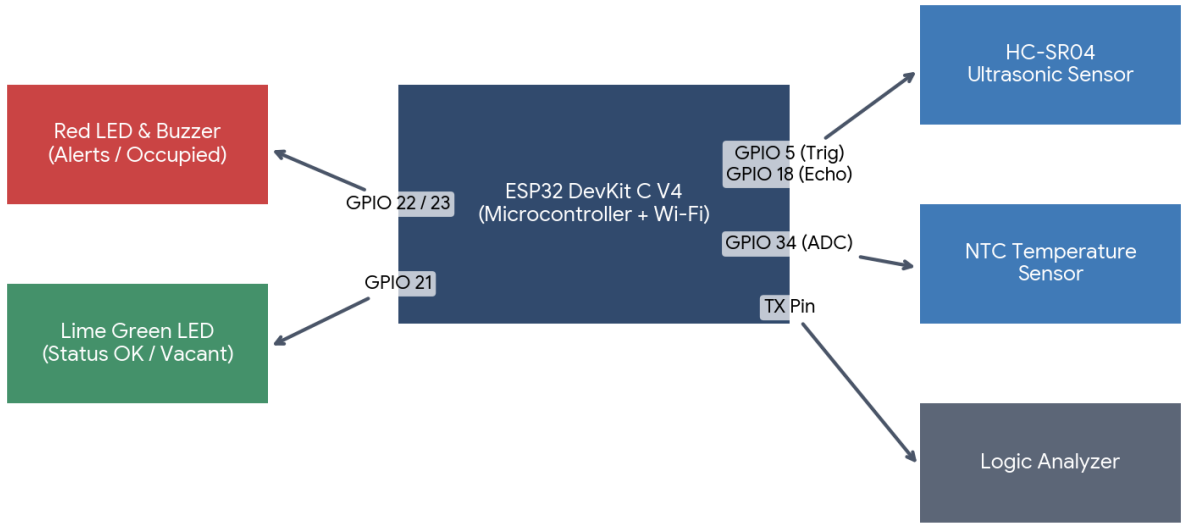

Figure 1: Unified Architectural Topography & System Pin Routes.

3. Hardware Interface & Pin Map Layout

Source Pin	Target Pin	Wire Group	System Functional Assignment
`esp:5`	`ultrasonic1:TRIG`	Green	Digital Output: Fires high-frequency sound pulse sequences
`esp:18`	`ultrasonic1:ECHO`	Green	Digital Input: Receives returning wave time-of-flight windows
`esp:34`	`ntc1:OUT`	Green	Analog Input (ADC): Monitors voltage changes across the thermistor
`esp:22`	`led1:A (Red)`	Green	Digital Output: Drives local Stop/Occupied visual alarms
`esp:21`	`led2:A (Green)`	Green	Digital Output: Drives local Safe Clear/Vacant indicators
`esp:23`	`bz1:2`	Green	Digital Output: Modulates pulsing audio safety alerts
`esp:TX`	`logic1:D0`	Green	Diagnostics: Tracks serial packets for local validation routines

4. Functional Control Logic & Engineering Applications

A. Distance Variations based on Atmospheric Dynamics (Humidity & Temperature)

Ultrasonic distance calculation relies on the formula: Distance = (Velocity × Time-of-Flight) / 2. However, sound velocity (c) is heavily dependent on ambient air properties. In open parking decks, summer asphalt heating reduces air density, while shifting seasonal weather modifies relative humidity. If left uncompensated, fixed-velocity frameworks register massive tracking anomalies, causing false occupancy assertions or garage door collision errors.

```
Velocity of Sound (c) ≈ 331.4 + 0.61 × Temperature (°C) m/s
```

By reading raw analog gradients from the NTC thermistor, the ESP32 handles dynamic calculations within its processing stack, recalculating target parameters locally on every loop interval. This provides industrial-grade reliability across all weather variations.

B. Smart Parking Actuator State Machine

The edge firmware actively monitors tracking loops and dictates local signaling states across three operational tiers:

- **Vacant / Safe Approach (Distance > 30 cm):** The Limegreen LED is activated while the Red LED and Buzzer remain off, informing approaching drivers that the parking spot is free.
- **Perfect Zone Alignment:** Feedback scales confirm that the vehicle has settled squarely within its structural boundary targets.
- **Critical Proximity Alert / Stop Command (Distance < 30 cm):** The system cuts the green indicator, flashes the Red LED, and sounds a pulsing acoustic warning via the piezo buzzer to prevent structural collisions.

C. IoT Cloud Integration & Smart Infrastructure Real-World Scenarios

Integrating the WiFi.h framework shifts this circuit from a local tracking layout into an intelligent nodes system:

- **Municipal Management & Smart Signage:** Nodes stream live status packages over Wi-Fi, updating municipal boards to point incoming vehicles directly to clear slots, cutting congestion.
- **Automated Gating Control:** Compensated telemetry maps can connect via Wi-Fi protocols to trigger automatic gate openings or barrier locks based on verified proximity boundaries.
- **Predictive System Diagnostics:** Data packages can be sent continuously to tracking hubs, checking hardware accuracy via the logic line to spot maintenance needs before failures strike.

5. Integrated Firmware Implementation Code

```
#include <WiFi.h>

// Wi-Fi Connection Parameters
const char* ssid     = "Wokwi-GUEST";
const char* password = "";

// Pin Mappings
const int TRIG_PIN   = 5;
const int ECHO_PIN   = 18;
const int NTC_PIN    = 34;
const int RED_LED    = 22;
const int GREEN_LED  = 21;
const int BUZZER_PIN = 23;

// Calibration Parameters
const float DISTANCE_THRESHOLD = 30.0; // Critical boundary in centimeters

void setup() {
  Serial.begin(115200);

  // Interface Initializations
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Establish Wi-Fi Connection Protocol
  Serial.print("Connecting to Wi-Fi Network: ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("
Wi-Fi Link Established Successfully!");
  Serial.print("Assigned Node IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  // 1. Process Telemetry (Analog NTC Reading & Conversion)
  int rawADC = analogRead(NTC_PIN);
  float resistance = (4095.0 / (float)rawADC) - 1.0;
  resistance = 10000.0 / resistance;
  float temperature = resistance / 10000.0;
  temperature = log(temperature);
  temperature /= 3950.0;                        // Beta calculation mapping
  temperature += 1.0 / (25.0 + 273.15);
```

```
  temperature = 1.0 / temperature - 273.15;      // Conversion to Celsius

  // 2. Dynamic Wave Velocity Correction (Temperature & Humidity Adaptation)
  float soundSpeed = 331.4 + (0.61 * temperature);
  float speedCmPerMicro = soundSpeed * 0.0001;     // Scale to cm/microsecond

  // 3. Sensor Ping Cycle Execution
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);

  // 4. Calculate Distance Under Environmental Scaling
  float distance = (duration * speedCmPerMicro) / 2.0;

  // 5. System State Assessment & Log Transmission
  Serial.print("[IoT Telemetry] Temp: ");
  Serial.print(temperature, 1);
  Serial.print("°C | Sound Speed: ");
  Serial.print(soundSpeed, 1);
  Serial.print(" m/s | Compensated Distance: ");
  Serial.print(distance, 1);
  Serial.println(" cm");

  if (distance < DISTANCE_THRESHOLD) {
    // CRITICAL WARNING / BAY OCCUPIED BOUNDARY
    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);

    // Generate acoustic beep alarm pulse
    digitalWrite(BUZZER_PIN, HIGH);
    delay(100);
    digitalWrite(BUZZER_PIN, LOW);
  } else {
    // STANDARD ACCESSIBLE ROADWAY / BAY VACANT
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(BUZZER_PIN, LOW);
  }

  delay(1000); // 1-second operational sampling gap
}
```